

Algoritmy a grafy 1 (BI-AG1), Přednáška č. 6

Binární vyhledávací stromy, vyvažování

Dušan Knop, Michal Opler, Tomáš Valla

Fakulta informačních technologií
České vysoké učení technické v Praze
<https://courses.fit.cvut.cz/BI-AG1/>



ZS 2025/2026, 27. října 2025

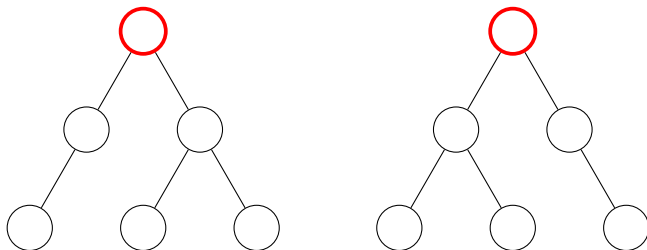
(Verze dokumentu: 14. 12. 2025 19:21)

Binární stromy (opakování)

Definice

Strom nazveme **binární**, pokud

- je zakořeněný,
- každý vrchol má nejvýše dva syny a
- u synů rozlišujeme, který je **levý** a který **pravý**.



Příklad dvou různých binárních stromů

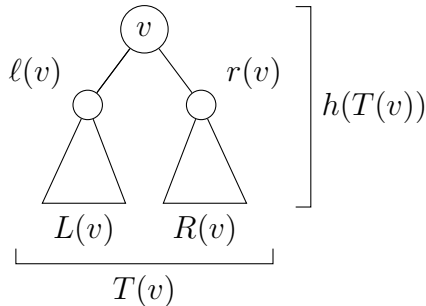
Značení binárních stromů

Značení

Pro vrchol v binárního stromu T značíme

- $\ell(v)$ a $r(v)$: levý a pravý syn vrcholu v ,
 - $L(v)$ a $R(v)$: levý a pravý podstrom vrcholu v ,
 - $p(v)$: otec vrcholu v ,
 - $T(v)$: podstrom tvořený kořenem v a všechny jeho potomky,
 - $h(T(v))$: hloubka stromu $T(v)$ je počet hladin $T(v)$,
 - $|T|$, $|T(v)|$, $|L(v)|$, $|R(v)|$: počet vrcholů stromu T , $T(v)$, $L(v)$ a $R(v)$.
-
- Pokud vrchol v nemá levého syna, položíme $\ell(v) = \emptyset$.
 - Podobně pro $r(v)$ a $p(v)$.
 - Pak dodefinujeme, že $T(\emptyset)$ je prázdný strom a $h(T(\emptyset)) = 0$.

Značení v binárním stromu



Konvence zápisu pseudokódu

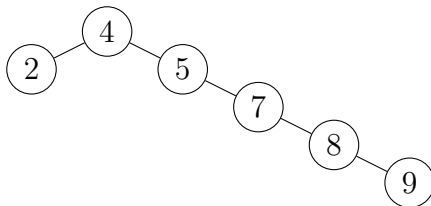
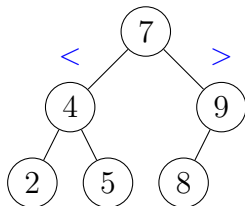
- Binární strom budeme v paměti počítače reprezentovat pomocí **spojových** struktur.
- V algoritmech popisujících v pseudokódu operace nad binárními stromy budeme pracovat s ukazateli na jejich vrcholy.

Definice binárního vyhledávacího stromu

Definice

Binární vyhledávací strom (BVS) je binární strom, v jehož každém vrcholu v je uložen unikátní klíč $k(v)$ a pro jehož každý vrchol v platí:

- Pokud $a \in L(v)$, pak $k(a) < k(v)$.
- Pokud $b \in R(v)$, pak $k(b) > k(v)$.



Příklady BVS

Operace podporované BVS

- **BVSShow**(v)
Vypiš vzestupně uspořádanou posloupnost klíčů všech vrcholů $T(v)$.
- **BVSMin**(v) (**BVSMax**(v))
Vrať vrchol obsahující minimální (maximální) klíč v $T(v)$.
- **BVSPred**(v, w) (**BVSSucc**(v, w))
Vrať předchůdce (následníka) vrcholu w v $T(v)$.
- **BVSFind**(v, x)
Vrať vrchol v $T(v)$ s klíčem x , pokud takový existuje.
- **BVSInsert**(v, x)
Vlož do $T(v)$ nový vrchol s klíčem x , pokud v něm ještě neexistuje.
- **BVSDelete**(v, x)
Odstraň z $T(v)$ vrchol s klíčem x , pokud takový existuje.

Výpis vzestupně uspořádaných klíčů BVS – `BVSShow( $v$ )`

Vstup: Ukazatel na kořen v nějakého BVS $T(v)$.

Výstup: Vzestupně uspořádaná posloupnost klíčů všech vrcholů v $T(v)$.

`BVSShow`(ukazatel na vrchol v)

(1) Pokud $v = \emptyset$: return $//T(v)$ je prázdný

(2) Zavoláme `BVSShow`($\ell(v)$)

(3) Vypíšeme $k(v)$

(4) Zavoláme `BVSShow`($r(v)$)

Z definice BVS a z definice **InOrder** průchodu binárním stromem plyne:

Pozorování

`BVSShow`(v) vypíše vzestupně uspořádané klíče vrcholů BVS $T(v)$ v čase $O(|T(v)|)$.

Nalezení vrcholu BVS s nejmenším klíčem – $BVSMin(v)$

Pozorování

Nejmenší klíč je **první** v posloupnosti vzestupně uspořádaných klíčů vygenerovaných v předchozím algoritmu InOrder průchodem a tudíž vrchol s nejmenším klíčem je v daném BVS **ten nejvíce vlevo**. Je to tudíž buď **list** nebo **vrchol s jedním synem**.

Vstup: Ukazatel na kořen v nějakého BVS $T(v)$.

Výstup: Ukazatel na vrchol obsahující nejmenší klíč v $T(v)$.

$BVSMin$ (ukazatel na vrchol v)

- (1) Pokud $\ell(v) = \emptyset$: vrať ukazatel na vrchol v
- (2) Jinak: vrať $BVSMin(\ell(v))$

Nalezení vrcholu s maximálním klíčem je analogické.

Hledání vrcholu BVS s daným klíčem –

$\text{BVSFind}(v, x)$

Vstup: Ukazatel na kořen v nějakého BVS $T(v)$ a klíč x .

Výstup: Ukazatel na vrchol s klíčem x , pokud takový v $T(v)$ existuje, jinak $\emptyset$.

$\text{BVSFind}(\text{ukazatel na vrchol } v, \text{klíč } x)$

- (1) Pokud $v = \emptyset$: vrať $\emptyset$
- (2) Pokud $x = k(v)$: vrať ukazatel na v
- (3) Pokud $x < k(v)$: vrať $\text{BVSFind}(\ell(v), x)$
- (4) Pokud $x > k(v)$: vrať $\text{BVSFind}(r(v), x)$

Pozorování

Korektnost plyne okamžitě z definice BVS: V libovolném vrcholu u platí, že všechny klíče v $L(u)$ jsou menší než $k(u)$ a všechny klíče v $R(u)$ jsou větší než $k(u)$.

Nalezení předchůdce vrcholu BVS – $BVSPred(v, w)$

Pozorování

- Klíč předchůdce prvku w v $T(v)$ je ve vzestupném výpisu klíčů algoritmem **BVSShow** **levým sousedem** $k(w)$.
- Pokud má w v $T(v)$ levý podstrom $L(w)$, pak je předchůdce w jeho maximem.
- V opačném případě je předchůdcem w první předek, do kterého vstoupíme **zprava** při průchodu nahoru.
- Evidentně, prvek s nejmenším klíčem předchůdce nemá, neboť nesplňuje ani jednu podmínku.

Tomu pak odpovídá následující pseudokód.

Nalezení předchůdce vrcholu BVS –

BVSPred(v, w)

Vstup: Ukazatel na kořen v nějakého BVS $T(v)$ a na nějaký jeho vrchol w .

Výstup: Ukazatel na předchůdce w v $T(v)$.

BVSPred(ukazatel na vrchol vrcholy v, w)

(1) Pokud $\ell(w) \neq \emptyset$: vrať BVSTreeMax($\ell(w)$)

(2) $z := p(w)$

(3) Dokud $z \neq \emptyset \ \& \ w = \ell(z)$:

(4) $w := z$

(5) $z := p(z)$

(6) vrať ukazatel na vrchol z

K zamyšlení:

Ověřte, že pro $w = BVSTreeMin(v)$ vrací tento algoritmus $\emptyset$.

Vložení vrcholu s daným klíčem do BVS

Vstup: Ukazatel na kořen v nějakého BVS $T(v)$ a klíč x .

Výstup: Ukazatel na kořen v BVS s prvkem s klíčem x .

BVSInsert(ukazatel na vrchol v , klíč x)

(1) Pokud $v = \emptyset$:

(2) vytvoř nový vrchol v s klíčem x

(3) vrať ukazatel na v a skončíme

(4) Pokud $x = k(v)$: nic

 //vrchol s klíčem x v $T(v)$ již existuje

(5) Pokud $x < k(v)$: $\ell(v) := \text{BVSInsert}(\ell(v), x)$

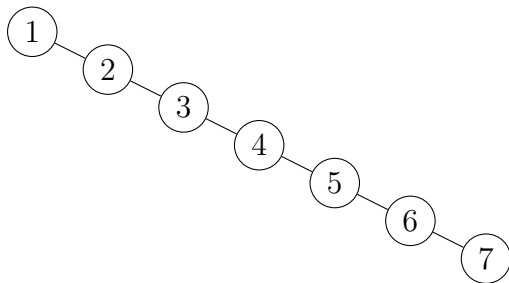
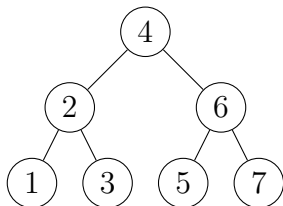
(6) Pokud $x > k(v)$: $r(v) := \text{BVSInsert}(r(v), x)$

(7) Vrať ukazatel na v

- Operaci **BVSInsert** lze chápat jako hledání daného klíče a pokud klíč nenajdeme, vložíme ho na **jednoznačně** určenou pozici jako nový list (předpokládáme unikátní klíče).

Tvar BVS

- Z dosud uvedeného vyplývá, že danou množinu klíčů bude existovat více korektních BVS lišících se tvarem, protože výsledný tvar BVS závisí nejen na hodnotách klíčů, ale i na pořadí, v jakém jsou do BVS vkládány.
- Operace **BVSShow** vrátí pro všechny možné BVS nad stejnou množinou klíčů stejnou uspořádanou posloupnost klíčů.
- Geometricky tomuto lineárnímu výpisu odpovídá průmět na vodorovnou osu.

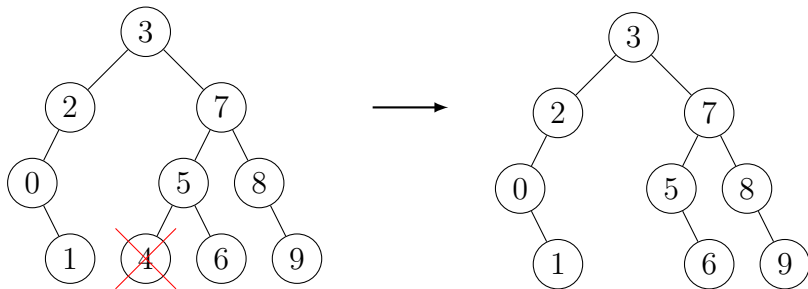


Odstranění vrcholu s daným klíčem z BVS I

- Nejdříve nalezneme vrchol, který obsahuje klíč, který chceme smazat.
- Při mazání vrcholu může nastat několik různých případů.

Případ 1) Pokud takový vrchol neexistuje, necháme strom beze změny.

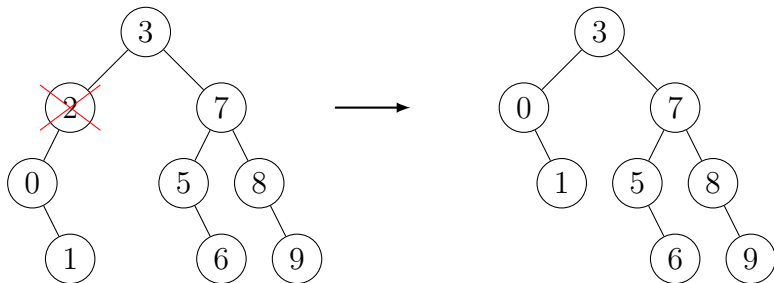
Případ 2) Jinak, pokud je nalezený vrchol **listem**, odtrhneme ho od stromu.



Odstranění listu BVS

Odstranění vrcholu s daným klíčem z BVS II

Případ 3) Má-li nalezený vrchol **jednoho syna**, nahradíme ho tímto synem.

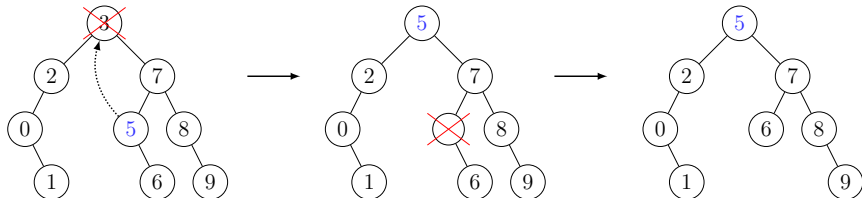


Odstranění vrcholu s jedním synem

Odstranění vrcholu s daným klíčem z BVS III

Případ 4) Pokud má nalezený vrchol, nazveme ho u , **dva syny**,

- pak u nemůžeme odstranit přímo, protože by nebylo kam připojit jeho syny.
- Využijeme ale faktu, že vrchol u má v BVS následníka $w = \text{BVSMIn}(r(u))$.
- A ten má nejvýše jednoho syna, viz Slajd 8.
- Klíč vrcholu u nahradíme klíčem vrcholu w .
- Aplikací postupu z případu 2) nebo 3) pak odstraníme z BVS vrchol w .



Odstranění vrcholu BVS se dvěma syny

BVSDDelete

Vstup: Ukazatel na kořen v nějakého BVS a klíč x .

Výstup: Ukazatel na kořen BVS, ze kterého byl odstraněn vrchol s klíčem x , pokud takový vrchol existoval.

BVSDDelete(ukazatel na vrchol v , klíč x)

- (1) Pokud $v = \emptyset$: vrať $\emptyset$
- (2) Pokud $x < k(v)$: $\ell(v) := \text{BVSDDelete}(\ell(v), x)$
- (3) Pokud $x > k(v)$: $r(v) := \text{BVSDDelete}(r(v), x)$
- (4) Pokud $x = k(v)$:
 - (5) Pokud $\ell(v) = r(v) = \emptyset$: vrať $\emptyset$
 - (6) Pokud $\ell(v) = \emptyset$: vrať $r(v)$
 - (7) Pokud $r(v) = \emptyset$: vrať $\ell(v)$
 - (8) $w := \text{BVSTMin}(r(v))$ // v má oba syny, w následník
 - (9) $k(v) := k(w)$
 - (10) $r(v) := \text{BVSDDelete}(r(v), k(w))$
- (11) vrať ukazatel na v

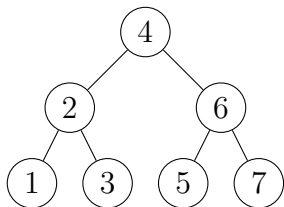
Vyváženost BVS

Pozorování

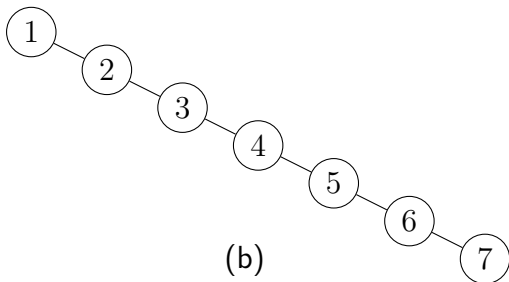
Časová složitost operací **BVSFind**, **BVSMIn**, **BVSInsert** a **BVSDelete** nad BVS $T(v)$ je $O(h(T(v)))$.

Hloubka $h(T(v))$ BVS $T(v)$ je

- (a) v nejlepším případě $\Omega(\log |T(v)|)$, ale
- (b) v nejhorším případě $O(|T(v)|)$ (např. BVS vytvořený postupným vložením klíčů $1, 2, \dots, n$ v tomto pořadí).



(a)



(b)

Dokonale vyvážený BVS

Definice

BVS nazveme **dokonale vyvážený**, pokud pro každý jeho vrchol v platí $||L(v)| - |R(v)|| \leq 1$.

Pozorování

Dokonale vyvážený BVS o velikosti n má $1 + \lfloor \log n \rfloor$ hladin a operace **BVSFind**, **BVSMin**, **BVSInsert** a **BVSDelete** na něm tedy mají časovou složitost $O(\log n)$.

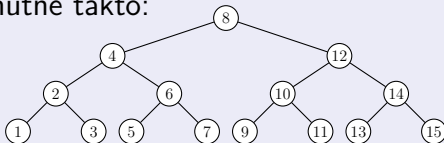
Věta (o složitosti operací nad dokonale vyváženým BVS)

Pokud má BVS zůstat po provedení operací **BVSInsert** a **BVSDelete** dokonale vyvážený, potom ať použijeme **jakoukoli implementaci** zmíněných operací, bude časová složitost aspoň jedné z nich $\Omega(n)$ pro nekonečně mnoho různých n .

Důkaz věty

Důkaz.

Uvažujme dokonale vyvážený BVS s klíči $1, 2, \dots, n$, kde $n = 2^k - 1$. Ten pak vypadá nutně takto:



- Kořen stromu je vždy prostřední z klíčů.
- Všechna lichá čísla jsou v listech stromu.
- Levý a pravý podstrom mají právě $2^{k-1} - 1$ vrcholů a jsou tedy opět určeny jednoznačně.
- ...

Dokončení důkazu.

- Nyní provedeme následující posloupnost **dvojic operací**:

$\text{Insert}(n+1), \text{Delete}(1), \text{Insert}(n+2), \text{Delete}(2), \dots$

- Po provedení i -té dvojice operací bude strom obsahovat hodnoty $i+1, \dots, i+n$.
- Podle toho, zda je i sudé nebo liché, se budou v listech nacházet buď všechna sudá, nebo všechna lichá čísla.
- Pokaždé se proto všem vrcholům změní, zda jsou listy, což ale nutně znamená upravit $\Omega(n)$ ukazatelů.
- Tedy aspoň jedna z operací BVInsert a BVDelete trvá $\Omega(n)$. □

Hloubkové vyvážení: AVL stromy

- Z předchozího výkladu tedy plyne, že žádné kritérium vyváženosti stejně jako příliš přísné kritérium vyváženosti vedou v nejhorším případě na lineární složitost alespoň jedné z operací BVS.
- Kritérium vyváženosti BVS je tedy třeba definovat méně restriktivně, stačí udržovat hloubku $O(\log n)$.

Definice

BVS je **hloubkově vyvážený**, pokud pro každý jeho vrchol v platí

$$|h(L(v)) - h(R(v))| \leq 1.$$

- Existuje několik různých řešení, zde si vysvětlíme tzv. **AVL-stromy**, které jsou efektivní implementací hloubkově vyvážených BVS.

Věta (o hloubce hloubkově vyvážených stromů)

Hloubkově vyvážený strom s n vrcholy má hloubku $\Theta(\log n)$.

Důkaz.

- Definujme $A_h = \min.$ počet vrcholů HV stromu hloubky h .
- Ukážeme, že A_h roste exponenciálně s h .
- Nejmenší takové stromy dané hloubky jsou maximálně „hloubkově vyvážené“ v mezích definice, čili v **každém** vnitřním vrcholu v platí $|h(L(v)) - h(R(v))| = 1$.
- Pro $h \leq 4$ lze nejmenší HV stromy hloubky h nalézt přímo:

$$A_1 = 1$$



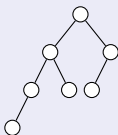
$$A_2 = 2$$



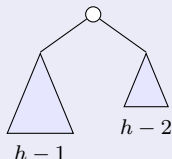
$$A_3 = 4$$



$$A_4 = 7$$



$$A_h = A_{h-1} + A_{h-2} + 1$$



- Obecně pak minimální HV strom hloubky h musí mít jako podstromy minimální HV stromy hloubky $h-1$ a $h-2$.
- Musí tedy platit $A_h = A_{h-1} + A_{h-2} + 1$.
- ...

Dokončení důkazu.

- Dokážeme indukcí, že $A_{h+1} \geq 2^{\frac{h}{2}} \doteq 1.41^h$.

IZ: $A_1 = 1 \geq 2^{0/2} = 1$ a $A_2 = 2 \geq 2^{\frac{1}{2}} \doteq 1,414$.

IK:

$$\begin{aligned} A_{h+1} &= 1 + A_h + A_{h-1} > \\ &> 2^{\frac{h-1}{2}} + 2^{\frac{h-2}{2}} = \\ &= 2^{\frac{h}{2}} \cdot (2^{-\frac{1}{2}} + 2^{-1}) > \\ &> 2^{\frac{h}{2}} \cdot \left(\frac{1}{2} + \frac{1}{2}\right) = \\ &= 2^{\frac{h}{2}}. \end{aligned}$$

- HV strom hloubky $h + 1$ má tedy **nejméně** $\sqrt{2}^h$ vrcholů.
- Proto HV strom o n vrcholech má hloubku **nejvýše** $\log_{\sqrt{2}}(n) + 1$.
- A protože je binární, nemůže mít hloubku menší než $\lfloor \log_2 n \rfloor$. viz Přednáška 4.
- Tedy hloubka HV stromu o n vrcholech je $\Theta(\log n)$. □

Operace musí udržovat hloubkové vyvážení

- Operace **BVSShow**, **BVSMin**, **BVSPred** a **BVSFind** nemění ani tvar ani obsah BVS, fungují tedy beze změny i pro AVL stromy.
- Operace vkládání a mazání prvků ale hloubky podstromů mění.
- Proto operace **AVLInsert** a **AVLDelete** vloží/vyjmou vrchol jako u **BVSInsert** a **BVSDelete**, ale pak kontrolují směrem ke kořenu hloubkovou vyváženost a při jejím porušení ji opravují.
- Jak kontrolovat a případně opravit hloubkovou nevyváženost?
- V každém vrcholu v vždy spočteme číslo

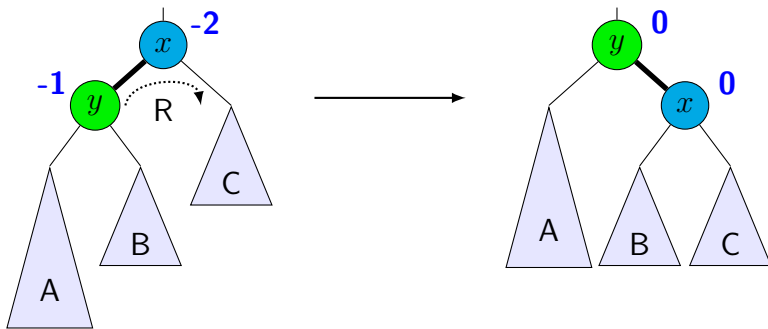
$$\delta(v) = h(R(v)) - h(L(v)),$$

které nazveme **znaménko vrcholu** v .

- V korektním AVL stromu může nabývat jen těchto hodnot:
 - ▶ $\delta(v) = +1$ (pravý podstrom hlubší) – značíme **+1**,
 - ▶ $\delta(v) = -1$ (levý podstrom hlubší) – značíme **-1**,
 - ▶ $\delta(v) = 0$ (oba podstromy stejně hluboké) – značíme **0**.
- Jakmile při kontrole narazíme na jiné $\delta(v)$, strom opravíme pomocí **jednoduchých** nebo **dvojitých rotací**.

Jednoduchá rotace doprava (R)

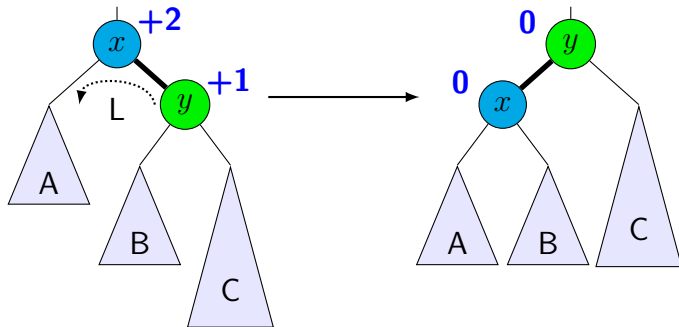
- Jednoduchá rotace **doprava** opraví hloubkovou nevyváženost v modelové situaci dle obrázku dole, kdy:
 - $y = \ell(x)$ a $h = h(B) = h(C) = h(A) - 1$ a tudíž
 - $\delta(y) = -1$ a $\delta(x) = -2$.
- Po provedení rotace doprava je $\delta(x) = \delta(y) = 0$.
- Z definice BVS plyne, že přepojení B od y k x zachovává BVS.



Jednoduchá rotace **doprava** v AVL stromu

Jednoduchá rotace doleva (L)

- Jednoduchá rotace **doleva** je zrcadlově symetrická modelová situace:
 - $y = r(x)$ a $h(A) = h(B) = h(C) - 1$ a tudíž
 - $\delta(x) = +2$ a $\delta(y) = +1$.
- Po provedení rotace doleva je opět $\delta(x) = \delta(y) = 0$.



Jednoduchá rotace **doleva** v AVL stromu

Jednoduchá rotace doleva (L)

Algoritmus `rotateLeft`(x)

Vstup: Vrchol x v BVS

- (1) $y := r(x)$
- (2) $r(x) := \ell(y)$
- (3) $\ell(y) := x$
- (4) $p(r(x)) := x$; $p(y) := p(x)$; $p(x) := y$;
- (5) $h(T(x)) := \max(h(L(x)), h(R(x))) + 1$;
- (6) $h(T(y)) := \max(h(R(y)), h(T(x))) + 1$;

Vnější algoritmus poté musí případnou změnu hloubky podstromu propagovat sám.

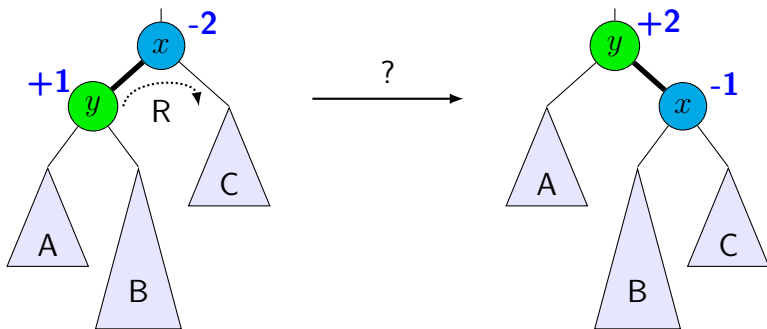
Pozorování

Pokud je x vrcholem v binárním vyhledávacím stromě, potom po volání `rotateLeft`(x) máme opět binární vyhledávací strom.

Vše symetricky pro `rotateRight`(x).

Situace, kdy jednoduchá rotace nepomůže

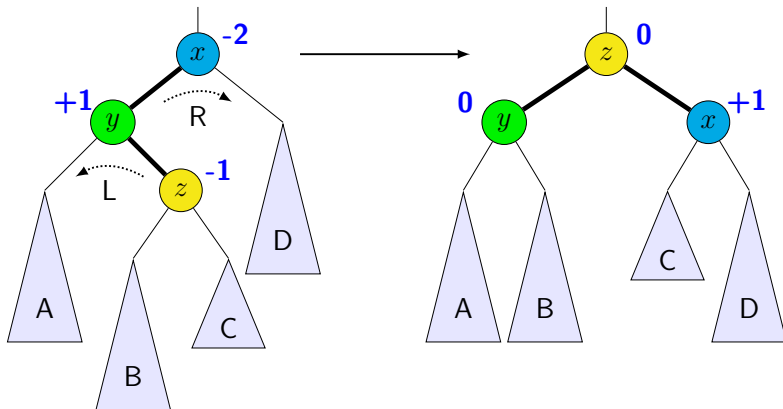
- Lze snadno nahlédnout, že jednoduchá rotace nepomůže v modelové situaci na obrázku, kdy $h(A) = h(C) = h(B) - 1$.
- Ukázáno pro rotaci doprava: místo $\delta(y) = +1$ a $\delta(x) = -2$ dostaneme $\delta(y) = +2$ a $\delta(x) = -1$.



- V podobných situacích je nutné použít **posloupnost dvou jednoduchých rotací**, buď LR nebo RL, které nazýváme **dvojitě rotace**.

Dvojitá rotace LR: modelová situace

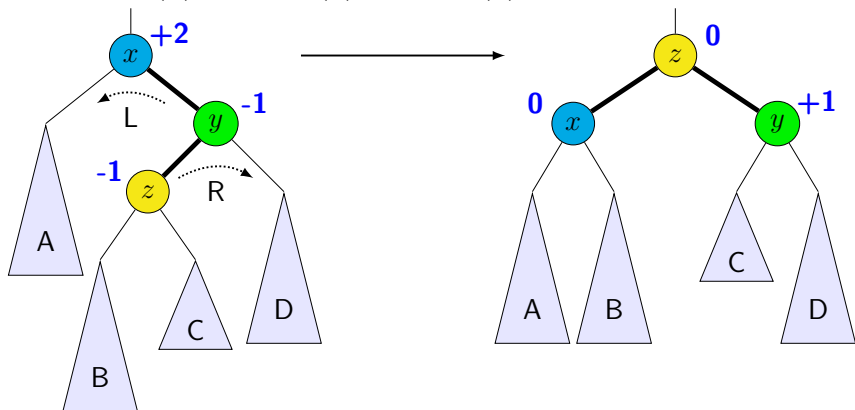
- Nechť $h(A) = h(B) = h(D) = h(C) + 1$.
- A tedy $\delta(x) = -2$, $\delta(y) = +1$ a $\delta(z) = -1$.



- Po provedení LR rotace máme $\delta(y) = \delta(z) = 0$ a $\delta(x) = +1$.
- Funguje zjevně i pro situace, kde je na počátku $\delta(z) = +1$ nebo $\delta(y) = +2$.

Dvojitá rotace RL: modelová situace

- Nechť opět symetricky $h(A) = h(B) = h(D) = h(C) + 1$.
- A tedy $\delta(x) = +2$, $\delta(y) = -1$ a $\delta(z) = -1$.



- Po provedení RL rotace máme $\delta(x) = \delta(z) = 0$ a $\delta(y) = +1$.
- Funguje zjevně, i když je na počátku $\delta(z) = +1$ nebo $\delta(y) = -2$.

AVLInsert

- Nový vrchol vložíme standardně jako list se znaménkem **0**.
- Z prázdného podstromu hloubky 0 se tak stal jednovrcholový podstrom hloubky 1.
- Potom je třeba přepočítat hloubky stromů na cestě z jeho rodiče do kořene.
- Proto budeme propagovat nahoru informaci o tom, že se **zvětšila** hloubka podstromu.
- V jednotlivých úrovních se tato informace zpracuje v závislosti na hloubkách příslušných sourozenců.
- Tuto kontrolu a případné napravení nevyvážeností můžeme elegantně provést během návratu z rekurze v proceduře **AVLInsert**.

AVLInsert – oprava hloubkového vyvážení

- Popíšeme propagaci pro jeden vrchol.
- Nechť do nějakého vrcholu x přišla z jeho syna informace o prohloubení podstromu.
- Ukážeme pro případ, kdy přišla z levého syna.
- Případ, kdy přišla z pravého syna, se řeší obdobně, jen se vymění význam **+1** a **-1**, levého a pravého podstromu a směr rotací.
- Rozlišíme **tři** případy podle znaménka vrcholu x .

AVLInsert: oprava hloubk. vyvážení, případ 1)

1) Vrchol x měl znaménko **+1**.

- ▶ Hloubka levého podstromu se právě vyrovnala s hloubkou pravého, čili znaménko x se změní na **0**.
- ▶ Hloubka podstromu $T(x)$ se ale **nezměnila**, takže propagování informace **zastavíme**.

AVLInsert: oprava hloubk. vyvážení, případ 2) a 3)

2) Vrchol x měl znaménko **0**.

- ▶ Znaménko x se změní na **-1**.
- ▶ Hloubka podstromu $T(x)$ se zvětšila o jedna, takže musíme **pokračovat** v propagování.

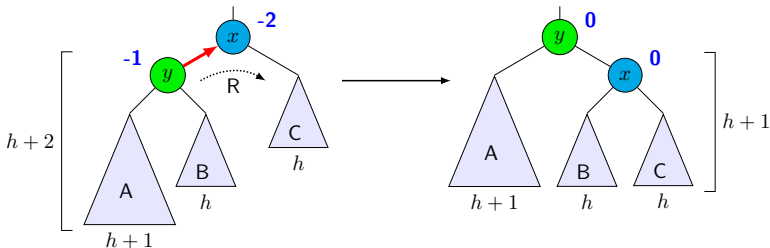
3) Vrchol x měl znaménko **-1**, tedy teď získá $\delta(x) = \mathbf{-2}$, je třeba vyvažovat.

- ▶ Označme y vrchol, z něž přišla informace o prohloubení, čili levého syna vrcholu x .
- ▶ Rozebereme opět **tři** případy podle znaménka vrcholu y .

AVLInsert: oprava hloubk. vyvážení, případ 3a)

3a) Vrchol y má znaménko **-1**.

- ▶ Označíme-li h hloubku podstromu C , podstrom $T(y)$ má hloubku $h + 2$, takže podstrom A má hloubku $h + 1$ a podstrom B hloubku h .
- ▶ Provedeme jednoduchou rotaci R hrany $\{x, y\}$ (viz Slajd 26).



- ▶ Tím získá vrchol x znaménko **0**, podstrom $T(x)$ hloubku $h + 1$, vrchol y znaménko **0** a podstrom $T(y)$ hloubku $h + 2$.
- ▶ Jelikož před započítáním operace **AVLInsert** měl podstrom $T(x)$ hloubku $h + 2$, z pohledu vyšších pater se nic nezměnilo. Propagování (červená šipka) tedy **zastavíme**.

AVLInsert: oprava hloubk. vyvážení, případ 3b)

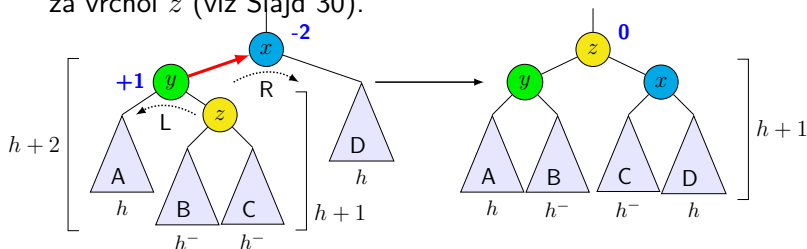
3b) Vrchol y má znaménko $+1$.

- ▶ Označíme z pravého syna vrcholu y (musí existovat).
- ▶ Označíme jednotlivé podstromy tak jako na obrázku a spočítáme jejich hloubky.

Referenční hloubku h zvolíme podle podstromu D .

Hloubky h^- znamenají „buď h nebo $h - 1$ “.

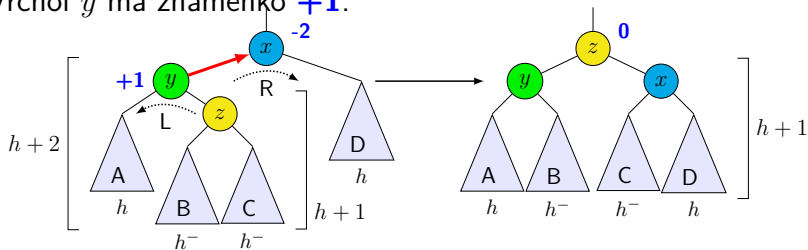
- ▶ Provedeme dvojitou LR rotaci, která celou konfiguraci překoření za vrchol z (viz Slajd 30).



- ▶ Přepočítáme hloubky a znaménka.
- ▶ ...

AVLInsert: oprava hloubk. vyvážení, případ 3b)

3b) Vrchol y má znaménko $+1$.



- ▶ Vrchol x bude mít znaménko buď 0 nebo $+1$, vrchol y buď -1 nebo 0 , každopádně oba podstromy $T(x)$ a $T(y)$ získají hloubku $h + 1$.
- ▶ Proto vrchol z získá znaménko 0 .
- ▶ Před započítáním **AVLInsertu** činila hloubka celé konfigurace $h + 2$, nyní je také $h + 2$, takže propagování **zastavíme**.

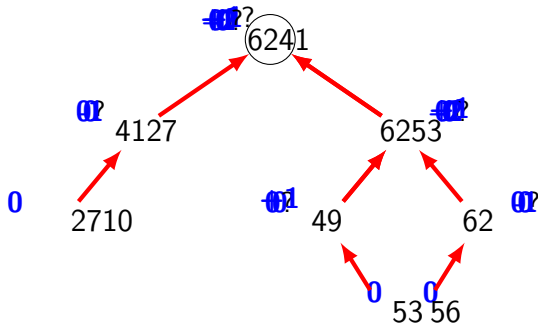
AVLInsert: oprava hloubk. vyvážení, případ 3c)

3c) Vrchol y má znaménko 0.

- ▶ Tento případ nemůže nikdy nastat, neboť z vrcholu se znaménkem 0 se informace o prohloubení v žádném z předchozích případů nešíří a nově přidaný list má sice znaménko 0, ale jeho otec nemůže mít nikdy -1.
- ▶ Čili zdola z vrcholu se znaménkem 0 nemůže informace o prohloubení přijít do vrcholu se znaménkem -1.

Ukázka vkládání do AVL stromu

AVLInsert(62)
AVLInsert(41)
AVLInsert(27)
AVLInsert(49)
AVLInsert(10)
AVLInsert(53)
AVLInsert(56)



Případ 2) Případ 3a) Případ
symetrický k 2) Případ 1) Případ 3b)

AVLDelete

- Obdobně jako **AVLInsert**.
- Vrchol smažeme podle **BVSDDelete** a po cestě zpět do kořene propagujeme informaci o **snížení** hloubky podstromu.
- Pokaždé mažeme list nebo vrchol s jediným synem, takže stačí propagovat od místa smazaného vrcholu nahoru.
- Popíšeme propagaci pro jeden vrchol.
- Nechť do vrcholu x přišla ze syna informace o snížení hloubky podstromu.
- Ukážeme opět pro případ, kdy přišla z levého syna.
- Případ, kdy přišla z pravého syna se řeší obdobně, jen se vymění význam **+1** a **-1**, levého a pravého podstromu a směr rotací.
- Rozlišíme opět **tři** případy podle znaménka vrcholu x .

AVLDelete: oprava hloubk. vyvážení, případ 1)

1) Vrchol x má znaménko **-1**.

- ▶ Hloubka levého podstromu se právě vyrovnala s hloubkou pravého, znaménko vrcholu x se mění na **0**.
- ▶ Hloubka podstromu $T(x)$ se snížila, takže **pokračujeme** v propagování.

AVLDelete: oprava hloubk. vyvážení, př. 2) a 3)

2) Vrchol x má znaménko **0**.

- ▶ Znaménko x se změní na **+1**.
- ▶ Hloubka podstromu $T(x)$ se nezměnila, takže propagování **zastavíme**.

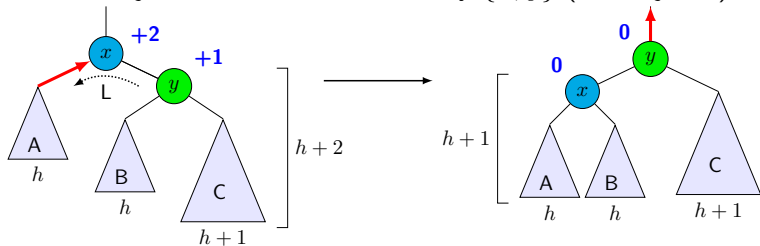
3 Vrchol x má znaménko **+1**.

- ▶ Tehdy se jeho znaménko změní na **+2** a musíme vyvažovat.
- ▶ Rozlišíme **tři** případy podle znaménka pravého syna y vrcholu x .

AVLDelete: oprava hloubk. vyvážení, případ 3a)

3a) Vrchol y má také znaménko **+1**.

- ▶ Označíme-li h hloubku podstromu A , bude mít $T(y)$ hloubku $h + 2$, takže C hloubku $h + 1$ a B hloubku h .
- ▶ Provedeme jednoduchou rotaci L hrany $\{x, y\}$ (viz Slajd 27).



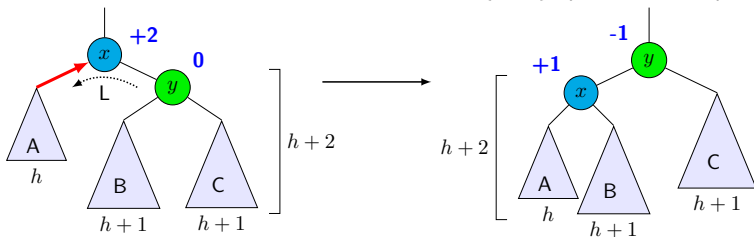
- ▶ Tím vrchol x získá znaménko **0**, podstrom $T(x)$ hloubku $h + 1$, takže vrchol y dostane také znaménko **0**.
- ▶ Před započítím **AVLDelete** měl podstrom $T(x)$ hloubku $h + 3$, nyní má $T(y)$ hloubku $h + 2$, takže z pohledu vyšších hladin došlo ke snížení hloubky.

Proto **pokračujeme** v propagování (červená šipka).

AVLDelete: oprava hloubk. vyvážení, případ 3b)

3b) Vrchol y má znaménko **0**.

- ▶ Nechť h je hloubka podstromu A . Pak $T(y)$ má hloubku $h + 2$ a B i C hloubku $h + 1$.
- ▶ Provedeme jednoduchou rotaci L hrany $\{x, y\}$ (viz Slajd 27).

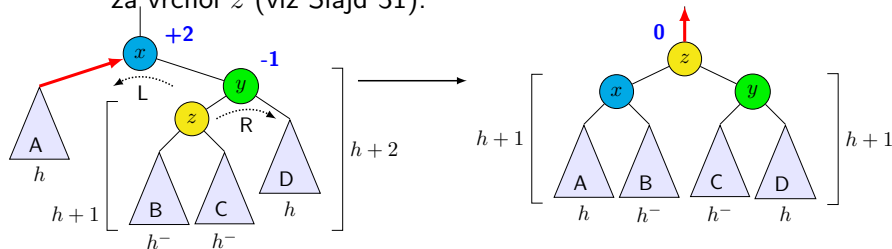


- ▶ Vrchol x získává znaménko **+1**, podstrom $T(x)$ hloubku $h + 2$, takže vrchol y obdrží znaménko **-1**.
- ▶ Hloubka podstromu $T(x)$ před začátkem **AVLDelete** činila $h + 3$, nyní má podstrom $T(y)$ hloubku také $h + 3$, a propagování **zastavíme**.

AVLDelete: oprava hloubk. vyvážení, případ 3c)

3c) Vrchol y má znaménko **-1**.

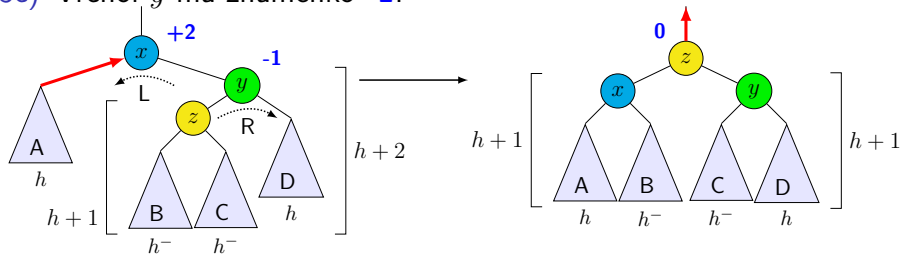
- ▶ Označíme z levého syna vrcholu y .
- ▶ Označíme podstromy podle obrázku a spočítáme jejich hloubky. Referenční hloubku h zvolíme opět podle A . Hloubky h^- znamenají „buď h nebo $h - 1$ “.
- ▶ Provedeme dvojitou RL rotaci, která celou konfiguraci překoření za vrchol z (viz Slajd 31).



- ▶ Přepočítáme hloubky a znaménka.
- ▶ ...

AVLDelete: oprava hloubk. vyvážení, případ 3c)

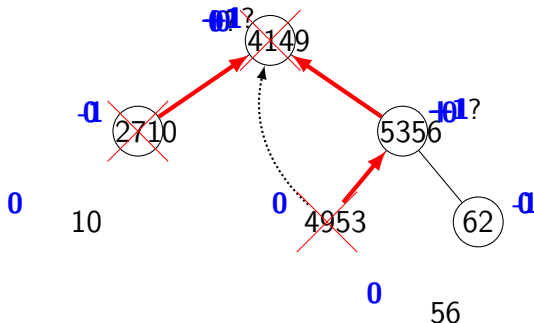
3c) Vrchol y má znaménko **-1**.



- ▶ Vrchol y bude mít znaménko buď **0** nebo **+1**, x buď **-1** nebo **0**. Podstromy $T(y)$ a $T(x)$ budou každopádně hluboké $h + 1$. Proto vrchol z obdrží znaménko **0**.
- ▶ Původní hloubka podstromu $T(x)$ před začátkem **AVLDelete** činila $h + 3$, nyní hloubka $T(z)$ činí $h + 2$, takže **pokračujeme** v propagování (červená šipka).

Ukázka mazání z AVL stromu

AVLDelete(41)
AVLDelete(27)



Případ 2) Případ 3c) Případ
symetrický k 1)

Pro dokonalé pochopení operací nad AVL stromy doporučuji skvělý
webový animátor AVL stromů na stránce

<https://www.cs.usfca.edu/~galles/visualization/AVLtree.html>.

Pozor – tento vizualizátor se liší od přednášky při mazání vrcholu se

Složitost operací

Tvrzení

AVLInsert a **AVLDelete** trvají $O(\log n)$.

Důkaz.

- Hloubka AVL stromu je vždy $\Theta(\log n)$.
- Původní implementace operací **BVSFind**, **BVSInsert** a **BVSDelete** tedy pracují v logaritmickém čase.
- Při vyvažování se operace vždy vrací po cestě do kořene a v každém vrcholu provede $\Theta(1)$ operací, takže celkově také trvá $\Theta(\log n)$. □

Co bychom po dnešku už měli znát

- Operace s binárním vyhledávacím stromem.
- Rotace v binárním vyhledávacím stromě, jednoduché a dvojité.
- AVL stromy a udržování hloubkové vyváženosti.